



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 21

Dr. Asad Mahmood

March 04, 2026

Lecture Outline

- Announcements (Assignment and Quiz 3)
- Outline of Previous Lecture:
 - **Chapter 3 – Arithmetic for Computers**
 - 3.5 Floating Point
- Outline to Today's Lecture:
 - **Quiz 2 Solution**
 - **Chapter 3 – Arithmetic for Computers**
 - 3.6 – onwards (self-read)

Chapter 3

Arithmetic for Computers

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading

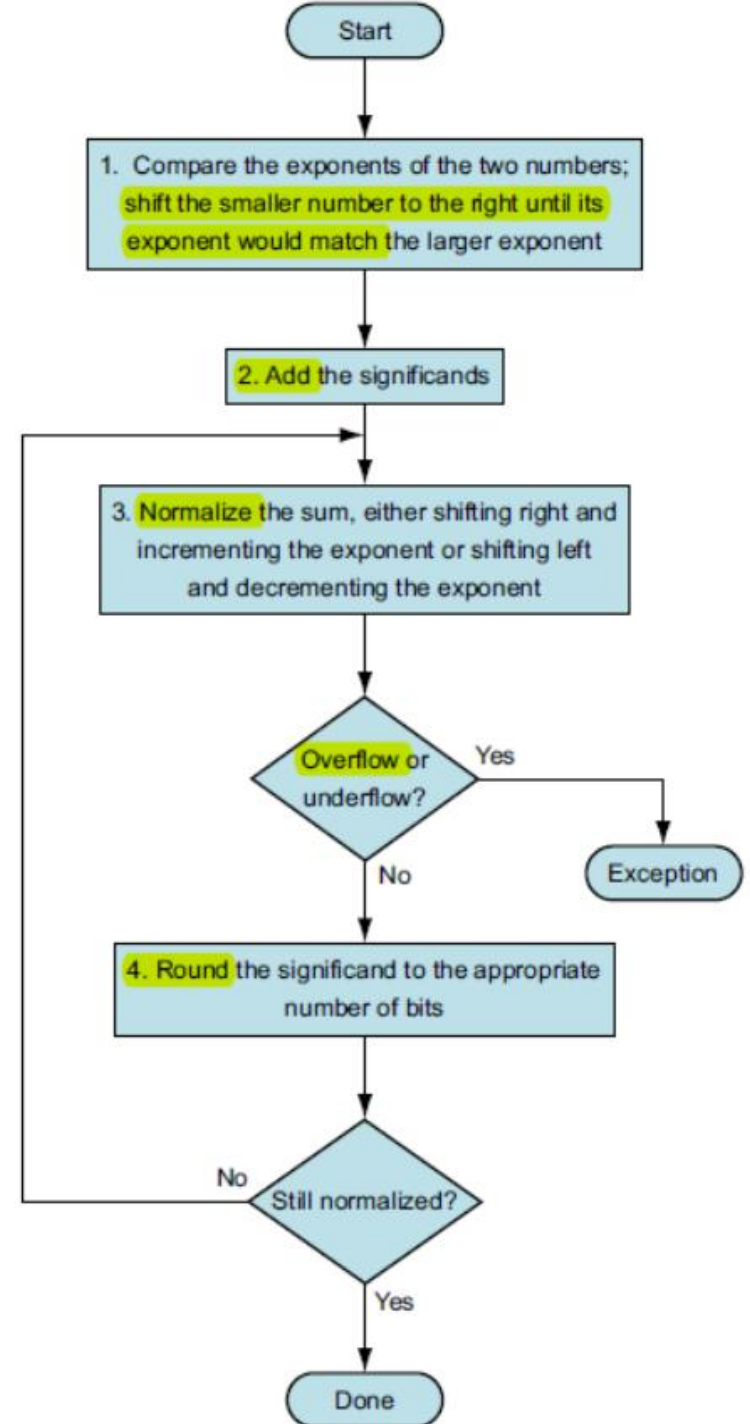
Floating-Point Addition

- How would we add decimal nos. in scientific notation?
- Process followed:
 1. Align decimal points
 - Shift number with smaller exponent
 2. Add significands
 3. Normalize result & check for over/underflow
 4. Round and renormalize if necessary

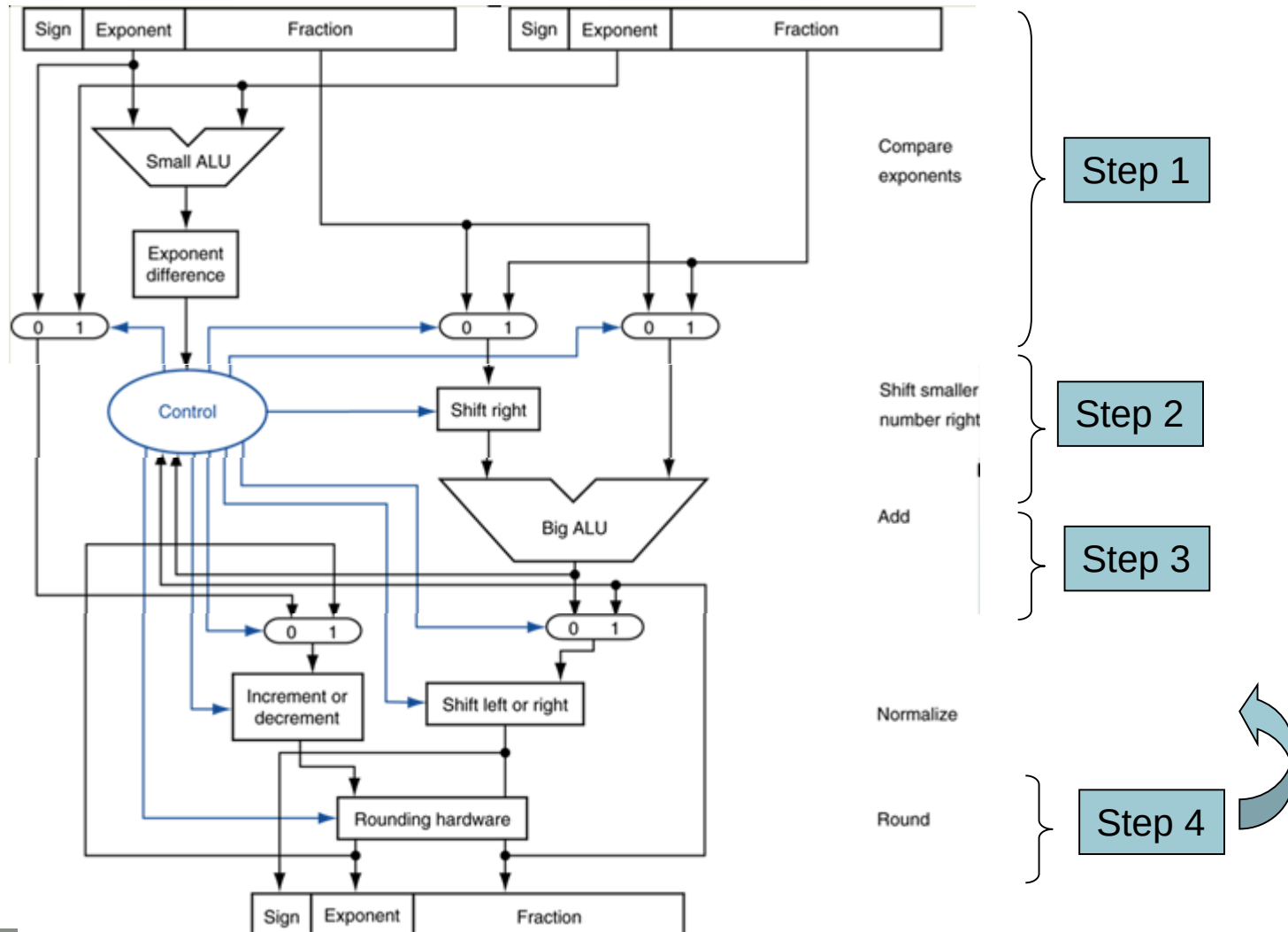
Floating-Point Addition

- Same process to follow for binary floating point addition:
 - Example: $(1.000_2 \times 2^{-1}) + (-1.100_2 \times 2^{-2})$
1. Align binary points
 2. Add significands
 3. Normalize result & check for over/underflow
 4. Round and renormalize if necessary

Floating-Point Addition



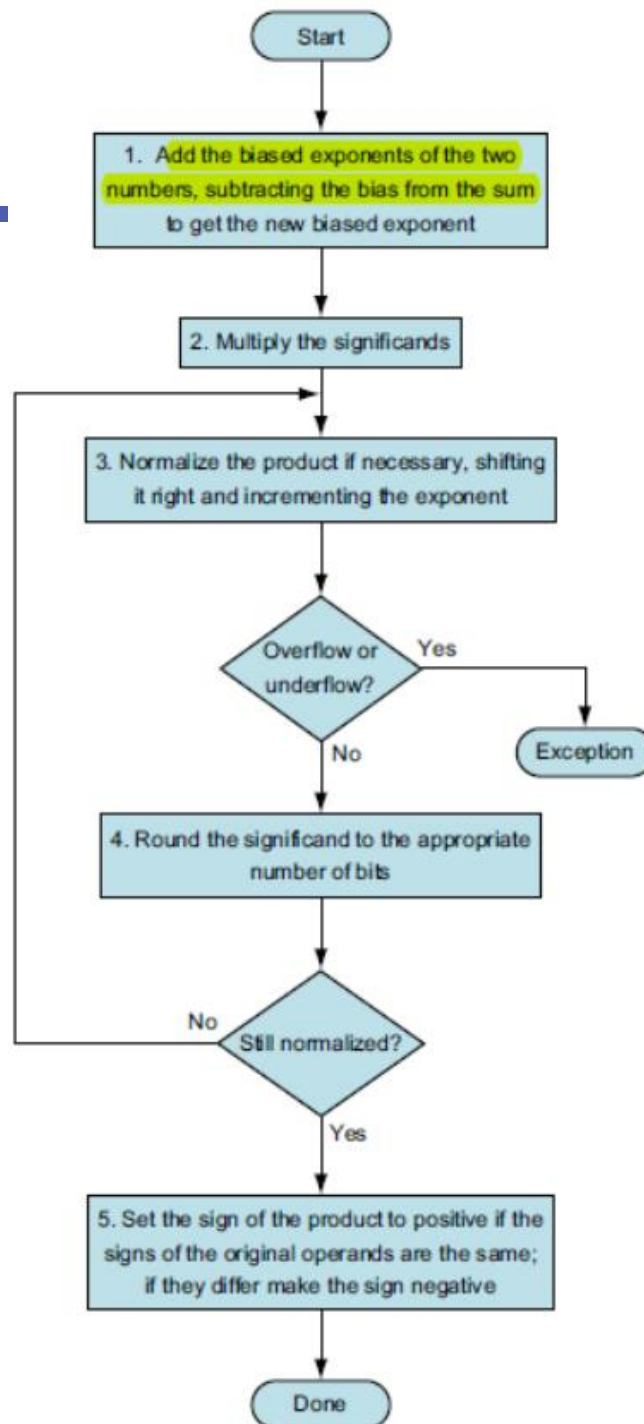
FP Adder Hardware



Floating-Point Multiplication

- How would we *multiply* decimal nos. in scientific notation?
- Process followed:
 1. Add exponents
 2. Multiply significands
 3. Normalize result & check for over/underflow
 4. Round and renormalize if necessary
 5. Determine sign of result from sign of operands

Floating-Point Multiplication



Floating-Point Multiplication

- Same process followed for multiplication of floating-point binary numbers
- Example: $(1.000_2 \times 2^{-1}) \times (-1.110_2 \times 2^{-2})$
- Process followed:
 1. Add exponents
 2. Multiply significands
 3. Normalize result & check for over/underflow
 4. Round and renormalize if necessary
 5. Determine sign of result from sign of operands

FP Arithmetic Hardware

- FP multiplier is of similar complexity to FP adder
 - But uses a multiplier for significands instead of an adder
- FP arithmetic hardware usually does
 - Addition, subtraction, multiplication, division, reciprocal, square-root
 - $FP \leftrightarrow$ integer conversion
- Operations usually takes several cycles
 - Can be pipelined

FP Instructions in RISC-V

- Separate FP registers: f0, ..., f31
 - double-precision
 - single-precision values stored in the lower 32 bits
- FP instructions operate only on FP registers
 - Programs generally don't do integer ops on FP data, or vice versa
- Benefit of dedicated FP registers?
- FP load and store instructions
 - flw, fld
 - fsw, fsd

FP Instructions in RISC-V

- Single-precision arithmetic
 - `fadd.s`, `fsub.s`, `fmul.s`, `fdiv.s`, `fsqrt.s`
 - e.g., `fadds.s f2, f4, f6`
- Double-precision arithmetic
 - `fadd.d`, `fsub.d`, `fmul.d`, `fdiv.d`, `fsqrt.d`
 - e.g., `fadd.d f2, f4, f6`
- Single- and double-precision comparison
 - `feq.s`, `flt.s`, `fle.s`
 - `feq.d`, `flt.d`, `fle.d`
 - Result is 0 or 1 in integer destination register
 - Use `beq`, `bne` to branch on comparison result

FP Example: °F to °C

- C code:

```
float f2c (float fahr) {  
    return ((5.0/9.0)*(fahr - 32.0));  
}
```

- fahr in f10, result in f10, literals in global memory space

- Compiled RISC-V code:

f2c:

```
f1w    f0,const5(x3)    // f0 = 5.0f  
f1w    f1,const9(x3)    // f1 = 9.0f  
fdiv.s f0, f0, f1       // f0 = 5.0f / 9.0f  
f1w    f1,const32(x3)   // f1 = 32.0f  
fsub.s f10,f10,f1       // f10 = fahr - 32.0  
fmul.s f10,f0,f10       // f10 = (5.0f/9.0f) * (fahr-32.0f)  
jalr   x0,0(x1)         // return
```

FP Example: Array Multiplication

- $C = C + A \times B$
 - All 32×32 matrices, 64-bit double-precision elements

- C code:

```
void mm (double c[][],
         double a[][], double b[][]) {
    size_t i, j, k;
    for (i = 0; i < 32; i = i + 1)
        for (j = 0; j < 32; j = j + 1)
            for (k = 0; k < 32; k = k + 1)
                c[i][j] = c[i][j]
                    + a[i][k] * b[k][j];
}
```

- Addresses of c, a, b in x10, x11, x12, and i, j, k in x5, x6, x7

FP Example: Array Multiplication

■ RISC-V code:

mm: . . .

```
        li    x28,32        // x28 = 32 (row size/loop end)
        li    x5,0          // i = 0; initialize 1st for loop
L1:     li    x6,0          // j = 0; initialize 2nd for loop
L2:     li    x7,0          // k = 0; initialize 3rd for loop
        slli  x30,x5,5       // x30 = i * 2**5 (size of row of c)
        add   x30,x30,x6     // x30 = i * size(row) + j
        slli  x30,x30,3      // x30 = byte offset of [i][j]
        add   x30,x10,x30    // x30 = byte address of c[i][j]
        fld   f0,0(x30)     // f0 = c[i][j]
L3:     slli  x29,x7,5       // x29 = k * 2**5 (size of row of b)
        add   x29,x29,x6     // x29 = k * size(row) + j
        slli  x29,x29,3      // x29 = byte offset of [k][j]
        add   x29,x12,x29    // x29 = byte address of b[k][j]
        fld   f1,0(x29)     // f1 = b[k][j]
```

FP Example: Array Multiplication

...

```
slli    x29,x5,5      // x29 = i * 2**5 (size of row of a)
add     x29,x29,x7     // x29 = i * size(row) + k
slli    x29,x29,3      // x29 = byte offset of [i][k]
add     x29,x11,x29    // x29 = byte address of a[i][k]
fld     f2,0(x29)      // f2 = a[i][k]
fmul.d  f1, f2, f1     // f1 = a[i][k] * b[k][j]
fadd.d  f0, f0, f1     // f0 = c[i][j] + a[i][k] * b[k][j]
addi    x7,x7,1        // k = k + 1
bltu    x7,x28,L3      // if (k < 32) go to L3
fsd     f0,0(x30)      // c[i][j] = f0
addi    x6,x6,1        // j = j + 1
bltu    x6,x28,L2      // if (j < 32) go to L2
addi    x5,x5,1        // i = i + 1
bltu    x5,x28,L1      // if (i < 32) go to L1
```

Accurate Arithmetic

- IEEE Std 754 specifies additional rounding control
 - Extra bits of precision (guard, round, sticky) for intermediate results before final rounding off
 - Choice of rounding modes
 - Allows programmer to fine-tune numerical behavior of a computation
- Example

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism (self-read)
- 3.7 Real Stuff: Streaming SIMD Extensions in x86 (self-read)
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply (self-read)
- 3.9 Fallacies and Pitfalls (self-read)
- 3.10 Concluding Remarks (self-read)
- 3.11 Historical Perspective and Further Reading (self-read)

Subword Parallelism

- Graphics and audio applications can take advantage of performing simultaneous operations on short vectors
 - Example: 128-bit adder:
 - Sixteen 8-bit adds
 - Eight 16-bit adds
 - Four 32-bit adds
- Also called data-level parallelism, vector parallelism, or Single Instruction, Multiple Data (SIMD)

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86 (self-read)
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply (self-read)
- 3.9 Fallacies and Pitfalls (self-read)
- 3.10 Concluding Remarks (self-read)
- 3.11 Historical Perspective and Further Reading (self-read)

x86 FP Architecture

- Originally based on 8087 FP coprocessor
 - 8×80 -bit extended-precision registers
 - Used as a push-down stack
 - Registers indexed from TOS: ST(0), ST(1), ...
- FP values are 32-bit or 64 in memory
 - Converted on load/store of memory operand
 - Integer operands can also be converted on load/store
- Very difficult to generate and optimize code
 - Result: poor FP performance

Streaming SIMD Extension 2 (SSE2)

- Adds 4 × 128-bit registers
 - Extended to 8 registers in AMD64/EM64T
- Can be used for multiple FP operands
 - 2 × 64-bit double precision
 - 4 × 32-bit double precision
 - Instructions operate on them simultaneously
 - Single-Instruction Multiple-Data

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86 (self-read)
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply (self-read)
- 3.9 Fallacies and Pitfalls (self-read)
- 3.10 Concluding Remarks (self-read)
- 3.11 Historical Perspective and Further Reading (self-read)

Matrix Multiply

■ Unoptimized code:

```
1. void dgemm (int n, double* A, double* B, double* C)
2. {
3.   for (int i = 0; i < n; ++i)
4.     for (int j = 0; j < n; ++j)
5.       {
6.         double cij = C[i+j*n]; /* cij = C[i][j] */
7.         for(int k = 0; k < n; k++ )
8.           cij += A[i+k*n] * B[k+j*n]; /* cij += A[i][k]*B[k][j] */
9.         C[i+j*n] = cij; /* C[i][j] = cij */
10.      }
11. }
```

Matrix Multiply

■ Optimized C code:

```
1.  //include <x86intrin.h>
2.  void dgemm (size_t n, double* A, double* B, double* C)
3.  {
4.      for ( size_t i = 0; i < n; i+=4 )
5.          for ( size_t j = 0; j < n; j++ ) {
6.              __m256d c0 = _mm256_load_pd(C+i+j*n); /* c0 = C[i][j] */
7.              for( size_t k = 0; k < n; k++ )
8.                  c0 = _mm256_add_pd(c0, /* c0 += A[i][k]*B[k][j] */
9.                      _mm256_mul_pd(_mm256_load_pd(A+i+k*n),
10.                                     _mm256_broadcast_sd(B+k+j*n)));
11.              _mm256_store_pd(C+i+j*n, c0); /* C[i][j] = c0 */
12.          }
13. }
```

Matrix Multiply

■ Optimized x86 assembly code:

<code>vmovapd (%r11),%zmm1</code>	<code># Load 8 elements of C into %zmm1</code>
<code>mov %rbx,%rcx</code>	<code># register %rcx = %rbx</code>
<code>xor %eax,%eax</code>	<code># register %eax = 0</code>
<code>vbroadcastsd (%rax,%r8,8),%zmm0</code>	<code># Make 8 copies of B element in %zmm0</code>
<code>add \$0x8,%rax</code>	<code># register %rax = %rax + 8</code>
<code>vfmadd231pd (%rcx),%zmm0,%zmm1</code>	<code># Parallel mul & add %zmm0, %zmm1</code>
<code>add %r9,%rcx</code>	<code># register %rcx = %rcx</code>
<code>cmp %r10,%rax</code>	<code># compare %r10 to %rax</code>
<code>jne 50 <dgemm+0x50></code>	<code># jump if not %r10 != %rax</code>
<code>add \$0x1,%esi</code>	<code># register %esi = %esi + 1</code>
<code>vmovapd %zmm1, (%r11)</code>	<code># Store %zmm1 into 8 C elements</code>

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86 (self-read)
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply (self-read)
- 3.9 Fallacies and Pitfalls (self-read)
- 3.10 Concluding Remarks (self-read)
- 3.11 Historical Perspective and Further Reading (self-read)

Right Shift and Division

- *Fallacy: Just as a left shift instruction can replace an integer multiply by a power of 2, a right shift is the same as an integer division by a power of 2.*
- Only for unsigned integers
- For signed integers
 - Arithmetic right shift: replicate the sign bit
 - e.g., $-5 / 4$
 - $11111011_2 \gg 2 = 11111110_2 = -2$
 - Rounds toward $-\infty$
 - c.f. $11111011_2 \ggg 2 = 00111110_2 = +62$